PSU FOODHUB

Prince sultan university - Collage of computer science

SE322 PROJECT	
DEEMA ALGHARIRI	222511260
NAJD ALLUHIDAN	222410194
FATIMAH ALMUNYIFI	221410859
MASHAEL ALJARBOU	221410453
LEEN ALSHAYA	221410456
HESSA ALMAZRUA	222410339

جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY

Table of Contents

Introduction 3

Overview 3

- **Purpose** 3
- **Scope** 3
- **References** 3

Architectural Background 3

- **The 4+1 View Model** 3
- **Architectural Goals** 4
- **Architectural Goals** 4

4+1 Views 4

- **User View (Scenarios)** 4
- **Logical View** 7
- **Logical View - Sequence Diagram** 8
- **Process View** 9
- **Development View** 11
- **Physical View** 11

Quality Attributes 13

1. **Security and Privacy** 13
2. **Performance** 14

.3 Availability 15

.4 Scalability 16

Architectural Pattern 17

1. **Selected Design** 17
2. **Why this style fits PSU FoodHub** 17
3. **How it is implemented in PSU FoodHub** 18

Software Design Documentation (SDD) 19

1. **Design Pattern 1: Observer Pattern** 19
2. **Design Pattern 2: Facade Pattern** 21
3. **Design Pattern 3: Builder Pattern** 23

Summary 25

Introduction

Overview

The process of ordering food on the PSU campus is a common source of queue frustration, the process of receiving incorrect orders, and inadequate service. Students and faculty lose and vendors record time and time waiting on orders during busy times. The PSU FoodHub application solves these problems with a smart mobile application to pre-order and pick food up.

Purpose

Using the 4+1 View Model, in this phase we will detail the FoodHub application architecture. The architecture balances the needs of the system to be scalable and maintainable with functional and non-functional requirements while also meeting the goals of a great user experience.

Scope

The core components of this architecture include:

- Mobile application frontend (Android and iOS)
- Order processing backend
- AI-based prediction engine
- Vendor dashboard
- Database and notification service

For the time being, the first deployment will be to PSU campus restaurants, although the architecture is designed with future growth in mind.

References

- SE322 Course Materials on Software Architecture
- 4+1 View Model Documentation
- Mobile Application Development Standards

Architectural Background

The 4+1 View Model

The 4+1 View Model is an example of an architectural framework that describes the design of software from five different vantage points. These include the Logical View, Process View, Development View, Physical View, and the Deployment View (which is sometimes referred to as the Use Case View). The Logical View describes the main abstractions and the relationships they

contain. The Process View describes the system's runtime behavior and concurrency The Development View describes the structure of the software modules. The Physical View describes the mapping of software onto the different hardware. The Scenario View describes how end users will interact with the different system components and how these pieces integrate with the rest of the components.

These different perspectives allow different stakeholders like developers, testers, and project managers to understand the system from their point of view.

Architectural Goals

- **G1**: Support 3000+ concurrent users during peak hours
- **G2**: Ensure system availability is 99.9% of the time
- **G3**: Improve abstraction to sub 2 seconds for order placement
- **G4**: Design the system to easily scale by adding new restaurants and vendors
- **G5**: Design the system to clearly maintain data security and user privacy
- **G6**: Provide clear seamless support for both platforms (Android and iOS)

Architectural Goals

- **C1**: The new system's authentication must use the PSU authorized authentication system.
- **C2**: The system must comply with Saudi Arabia's data protection regulations.
- **C3**: The system must integrate with PSU's payment system.
- **C4**: The new system will be limited to mobile platforms to access the system. There will be no desktop versions initially.
- **C5**: The system must work within the existing infrastructure of the network in PSU.
- **C6**: The development of the system must use open-source technologies when it is possible.

4+1 Views

User View (Scenarios)

Scenario 1: Student Food Ordering Process

Actor: PSU Student

1. Student launches the FoodHub application and signs in using the PSU account.
2. The student browses through the various restaurants available on campus.
3. The student selects “Burger Palace” and examines the menu.
4. The student customizes a burger by adding extra cheese and removing the onions.
5. The student sees a predicted time of preparation by the AI (8-12 minutes).



6. The student chooses the best time to pick up the burger (12:15 PM) and makes a payment using the saved card.
7. The student receives an order confirmation and the student can track the order in real time.
8. The student receives a notification when the order is ready for pickup.

Scenario 2: Vendor Order Management

Actor: Restaurant Staff

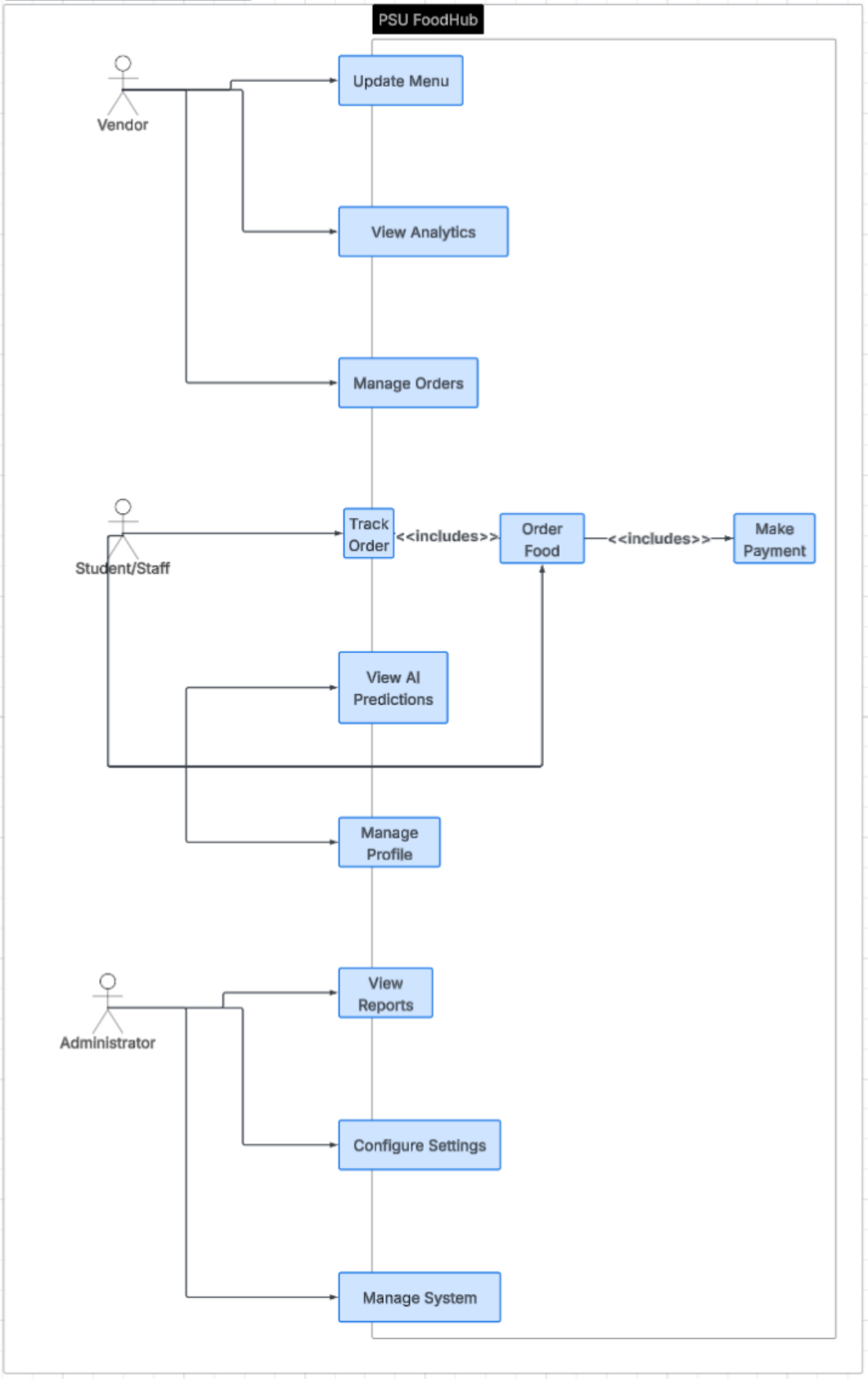
1. The vendor logs on to the dashboard with a secure username and password.
2. The vendor sees the incoming orders in the queue in real time.
3. The vendor sees AI prediction for the peak hours (they will be highlighted in red).
4. The vendor changes the status of the order from “Preparing” to “Ready”
5. The vendor pushes a notification to the student.
6. The vendor checks the daily summary on the analytics for orders and the time taken for preparation.

Actor: System Administrator

1. Logs into the admin portal with higher privileges
2. Views system health status
3. Adds a new restaurant vendor on the platform
4. Sets vendor permissions and menu upload restrictions
5. Checks AI prediction accuracy metrics
6. Creates weekly usage reports for PSU management

جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY

PSU FoodHub Use Case Diagram

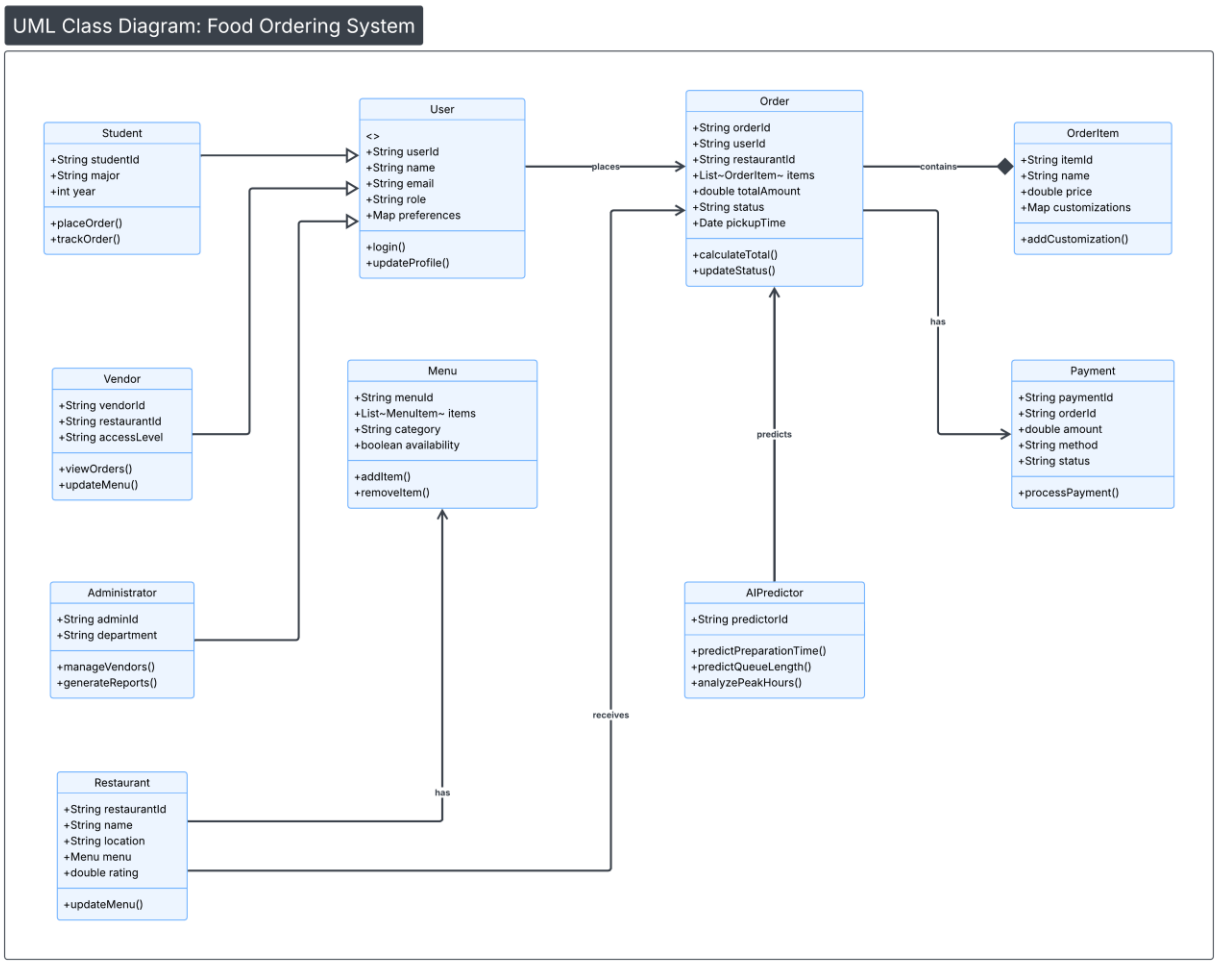


Logical View

Description:

The Logical View depicts the main system's abstractions and interconnections in the PSU FoodHub system. It consists of:

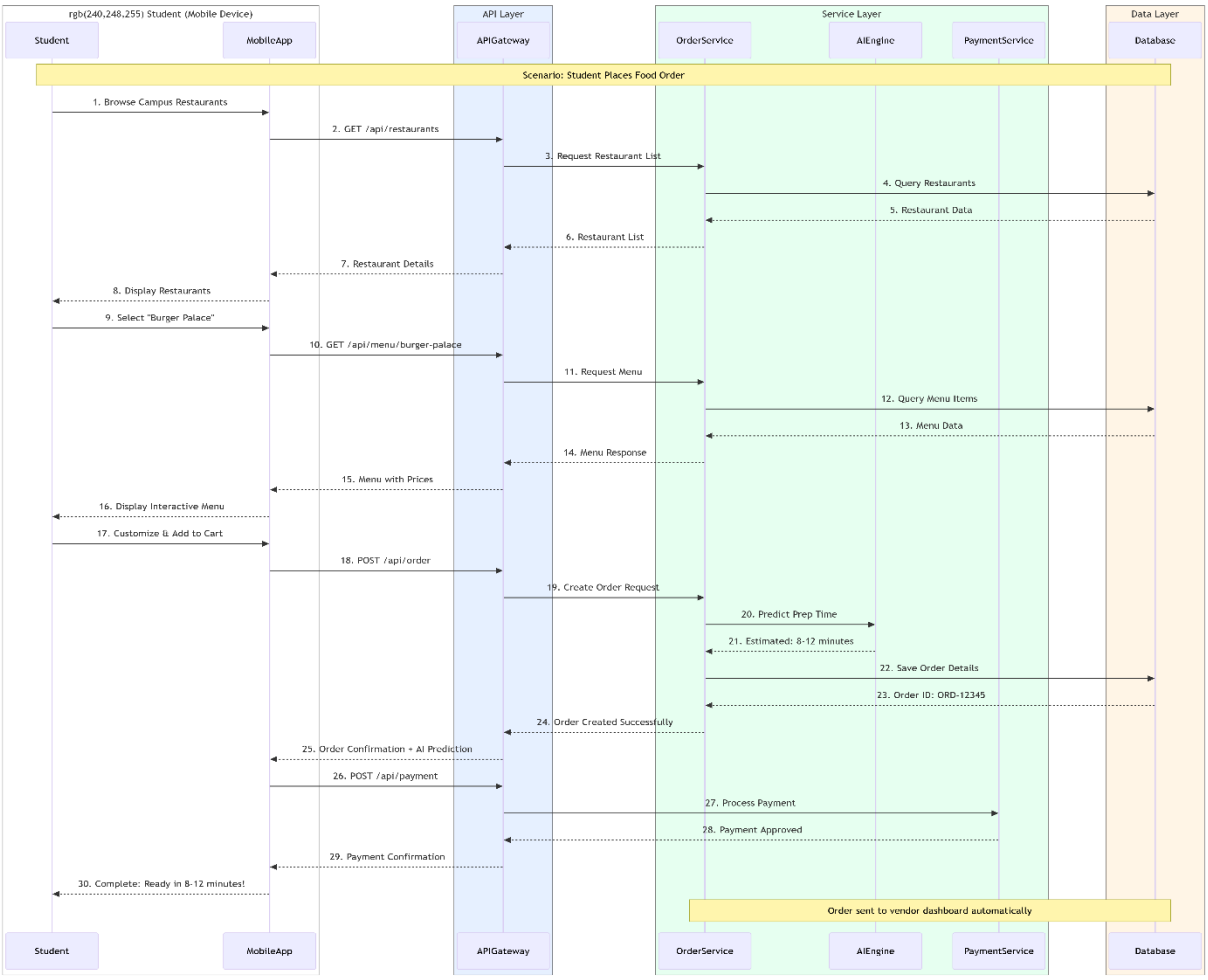
- **User Hierarchy**: A high-level User abstract class with subclasses (Student, Vendor, Administrator)
- **Major Components**: Order, OrderItem, Restaurant, Menu, Payment
- **AI Element**: AIPredictor service for time and queue prediction
- **Relationships**: Descriptions of how the entities (places, contains, receives, has, predicts) interact with other entities.



Logical View - Sequence Diagram

Description:

- The sequence diagram describes the interaction flow of a student placing a food order, which includes:
- Time sequence for order placement messages between different components of the system
- Layered architecture: Mobile App → API Gateway → Services → Database
- Important order placement activities: browsing the menu, creating an order, making a payment, and AI prediction
- Order tracking and AI prediction in real-time



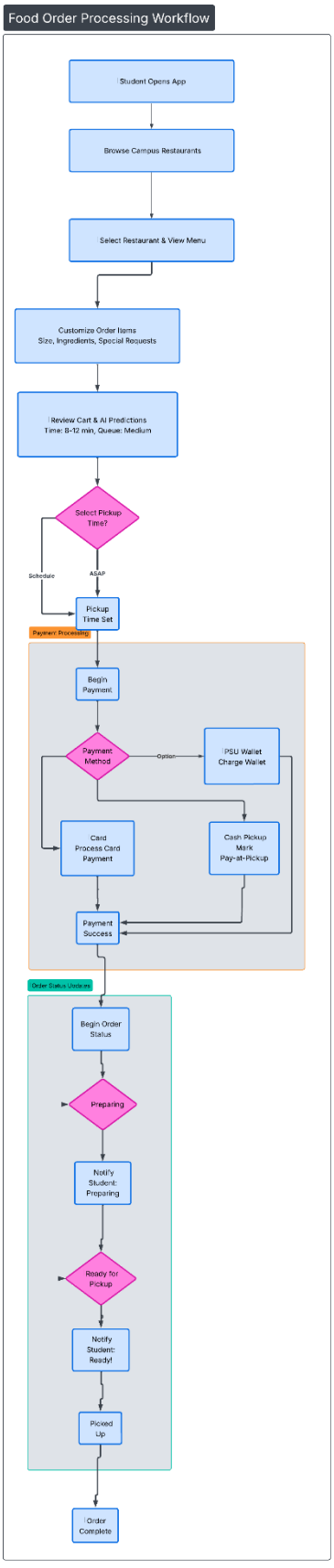
Process View

Description:

The Process View includes the steps involved in order processing from beginning to end, including:

- Primary flow from app launch to order completion
- Decisions: Payment method, order status updates
- Concurrent activities: Student notifications and vendor prep
- Mistake management: Different scenario alternative paths

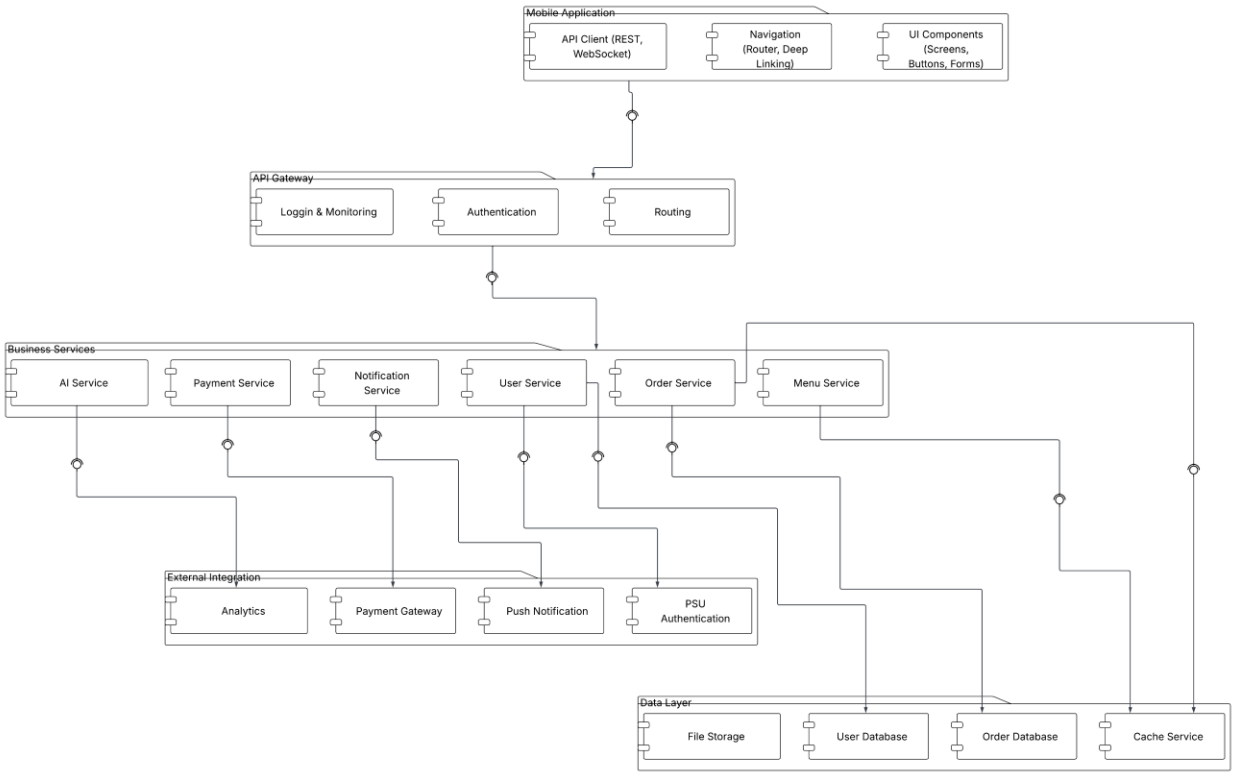
جامعة الأمير سلطان
PRINCE SULTAN UNIVERSITY



Development View

Overview:

1. The Development View illustrates the modular design of the system and the relationships between its components. It structures the system into:
2. Layered architecture: Mobile App → API Gateway → Business Services → Data Layer
3. Microservices: separated services for orders, menus, users, payments, AI, and notifications
4. Dependencies: interactions and interrelations among components
5. External integrations: links to PSU systems and other external services.

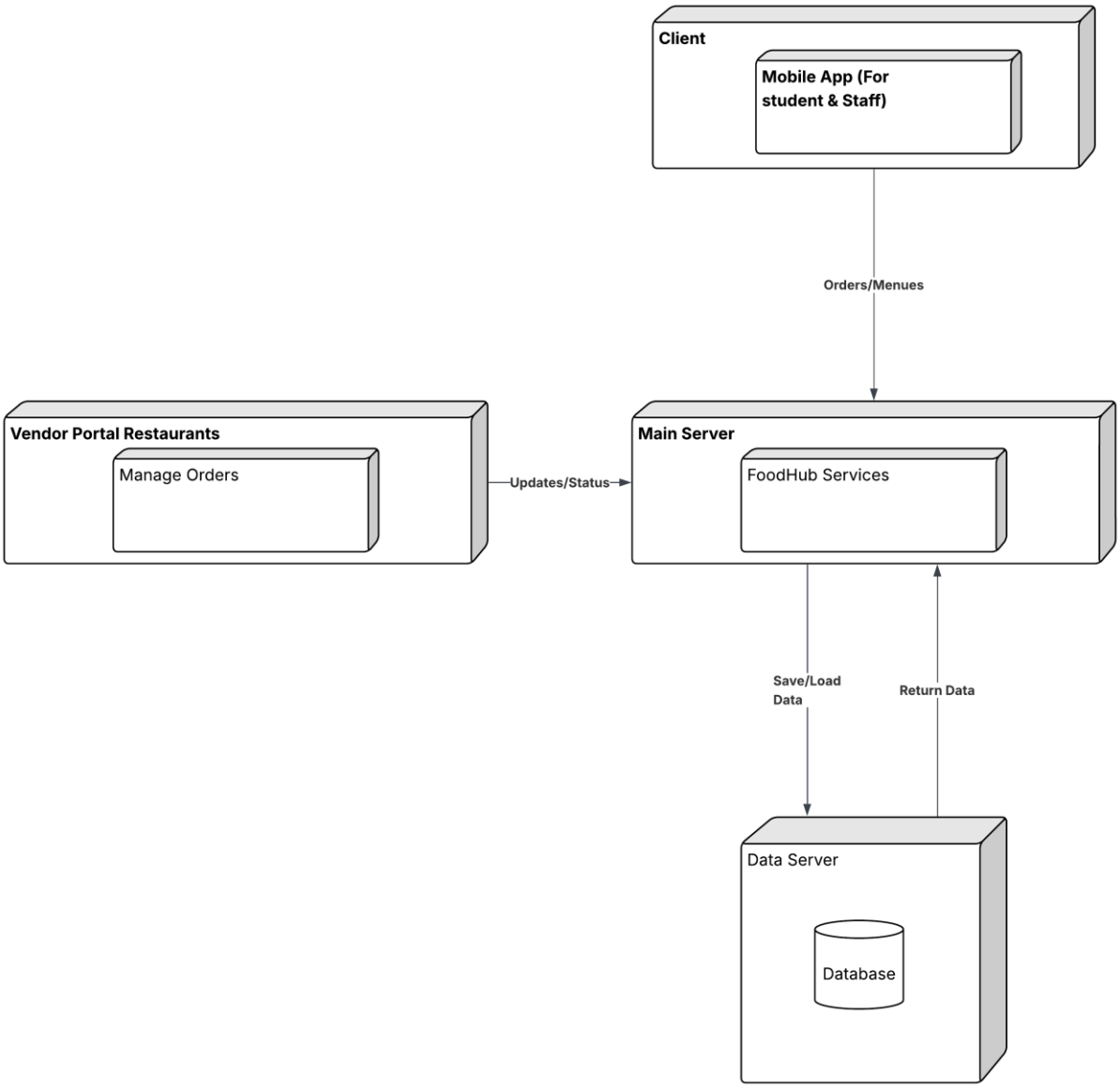


Physical View

The Physical View shows how the system components are deployed on hardware infrastructure. It includes:

- Client devices: Different types of user devices
- Server infrastructure: Load balancers, application servers, database servers
- Specifications: CPU, RAM, storage configurations

- Network connections: HTTPS connections, internal networking
- External systems: Integration with PSU and third-party services



Quality Attributes

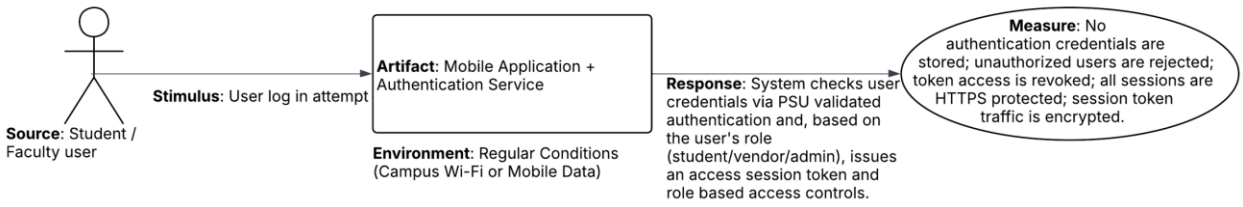
We will describe the structure of each scenario as follows: Source, Stimulus, Environment, Artifact, Response, and Response Measure.

1. Security and Privacy

Security is one of the focused elements of PSU FoodHub. This is due to the PSU FoodHub system processing student data, PSU system authentication identifiers, and payment info. The system must have safe log ins, use expensive billing for data, and use a form of communication that is based on roles, so that only authorized users (students, vendors, admins) can access features that are authorized for them. Thwarting the accessing of data and the unauthorized use of data regulated by the university and within the country ensures and protects user trust.

Scenario S1: Secure Login via PSU Authentication

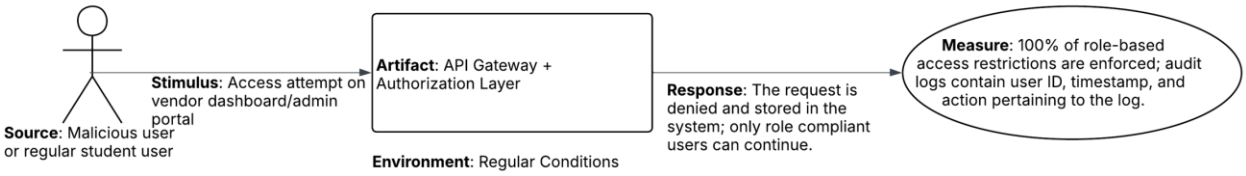
- **Source**: Student / Faculty user
- **Stimulus**: User log in attempt
- **Environment**: Regular Conditions (Campus Wi-Fi or Mobile Data)
- **Artifact**: Mobile Application + Authentication Service
- **Response**: System checks user credentials via PSU validated authentication and, based on the user's role (student/vendor/admin), issues an access session token and role based access controls.
- **Measure**: No authentication credentials are stored; unauthorized users are rejected; token access is revoked; all sessions are HTTPS protected; session token traffic is encrypted.



Scenario S2: Prevent Unauthorized Access to Vendor/Admin Functionality

- **Source**: Malicious user or regular student user
- **Stimulus**: Access attempt on vendor dashboard/admin portal
- **Environment**: Regular Conditions
- **Artifact**: API Gateway + Authorization Layer

- **Response**: The request is denied and stored in the system; only role compliant users can continue.
- **Measure**: 100% of role-based access restrictions are enforced; audit logs contain user ID, timestamp, and action pertaining to the log.

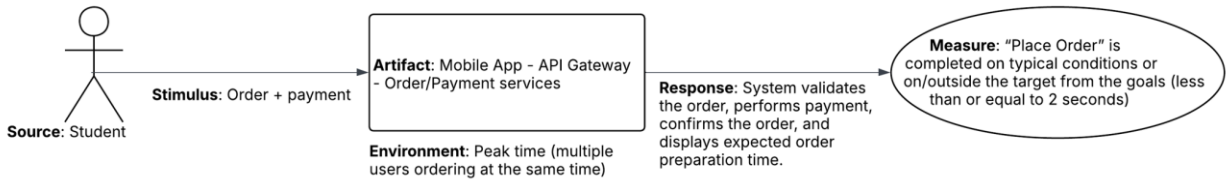


2. Performance

The PSU FoodHub system must also have a good performance, especially during the peak times of the lunch breaks. The system must have a good performance in order to process the order in a quick manner, have good performance for user interaction (goal performance within 2 seconds) and must have a performance to be able to track the order in real time. Optimized processing of the server and good performance in order communication provides the best user interaction for the user.

Scenario P1 - Fast food order

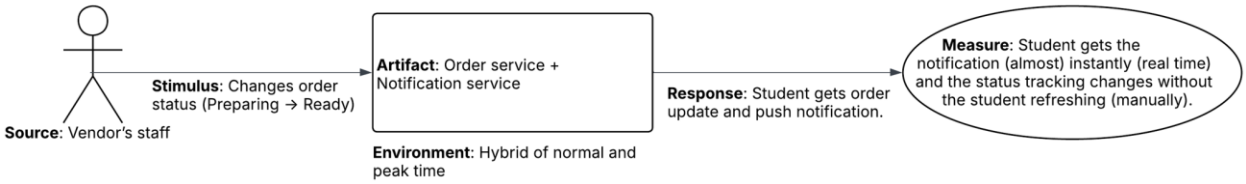
- **Source**: Student
- **Stimulus**: Order + payment
- **Environment**: Peak time (multiple users ordering at the same time)
- **Artifact**: Mobile App - API Gateway - Order/Payment services
- **Response**: System validates the order, performs payment, confirms the order, and displays expected order preparation time.
- **Measure**: “Place Order” is completed on typical conditions or on/outside the target from the goals (less than or equal to 2 seconds)



Scenario P2 - Real time order status updates

- **Source**: Vendor’s staff

- **Stimulus**: Changes order status (Preparing → Ready)
- **Environment**: Hybrid of normal and peak time
- **Artifact**: Order service + Notification service
- **Response**: Student gets order update and push notification.
- **Measure**: Student gets the notification (almost) instantly (real time) and the status tracking changes without the student refreshing (manually).

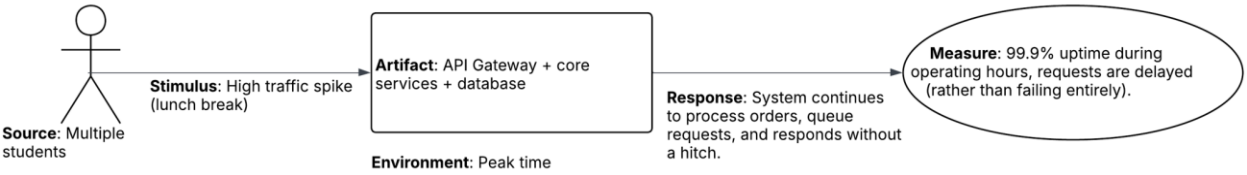


3. Availability

PSU FoodHub's system uptime target of 99.9% is essential to maintain campus operational hours. During that time, students are likely to be placing, tracking, and even canceling orders, which demonstrates the importance of consistent operational uptime. In the event that a non-core ordering functionality service (e.g., a notification service) ceases to operate, it is critical that the system continue to provide the core ordering service to students.

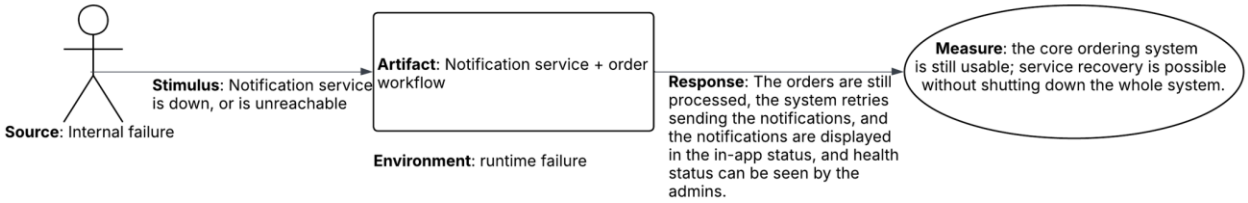
Scenario A1 - System availability during campus rush

- **Source**: Multiple students (concurrent users)
- **Stimulus**: High traffic spike (lunch break)
- **Environment**: Peak time
- **Artifact**: API Gateway + core services + database
- **Response**: System continues to process orders, queue requests, and responds without a hitch.
- **Measure**: 99.9% uptime during operating hours, requests are delayed (rather than failing entirely).



Scenario A2: Not everything worked (notifications service is down)

- **Source**: Internal failure
- **Stimulus**: Notification service is down, or is unreachable
- **Environment**: runtime failure
- **Artifact**: Notification service + order workflow
- **Response**: The orders are still processed, the system retries sending the notifications, and the notifications are displayed in the in-app status, and health status can be seen by the admins.
- **Measure**: the core ordering system is still usable; service recovery is possible without shutting down the whole system.

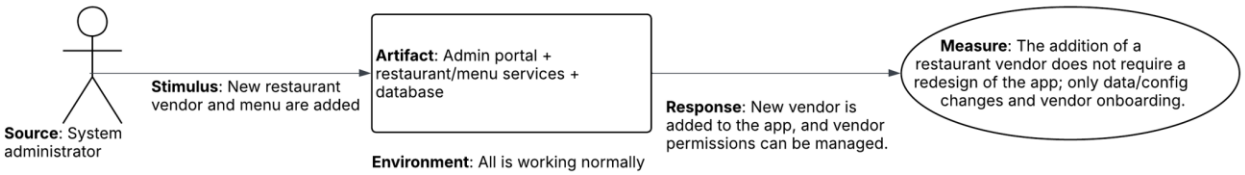


4. Scalability

PSU FoodHub's system design must accommodate and not impede growing user, restaurant, and order quantities. As more students and vendors use the system, it must offer the ability to horizontally scale by adding more service instances. This is critical to maintaining consistent demand and system performance during periods of expansion.

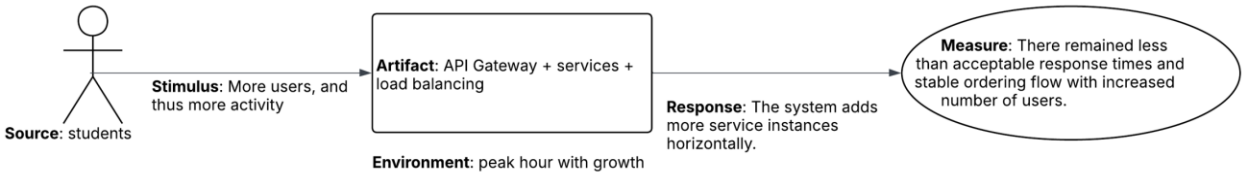
Scenario SC1: New restaurants can be added without system redesign

- **Source**: System administrator
- **Stimulus**: New restaurant vendor and menu are added
- **Environment**: All is working normally
- **Artifact**: Admin portal + restaurant/menu services + database
- **Response**: New vendor is added to the app, and vendor permissions can be managed.
- **Measure**: The addition of a restaurant vendor does not require a redesign of the app; only data/config changes and vendor onboarding.



Scenario SC2: Increased users is manageable

- **Source**: students
- **Stimulus**: More users, and thus more activity
- **Environment**: peak hour with growth
- **Artifact**: API Gateway + services + load balancing
- **Response**: The system adds more service instances horizontally.
- **Measure**: There remained less than acceptable response times and stable ordering flow with increased number of users.



Architectural Pattern

1. Selected Design

Client–Server + Layered + Microservices

As we already mentioned, our system has the following:

- Mobile App frontend
- API Gateway
- Individual services (orders, menus, users, payments, AI predictor, notifications)
- Databases are integrated with the external PSU

  This corresponds to Client–Server architecture with a Layered structure and Microservices at the backend.

2. Why this style fits PSU FoodHub

- **Performance**: API Gateway and services separation are the reasons for fast and focused processing, while keeping the order flow clean and optimized.
- **Availability**: When one service (notifications) goes down, the rest are still functional.
- **Security**: Using an API Gateway to centralize access is very helpful because it is easier to implement authentication, authorization, and access logging.
- **Scalability**: Services can be scaled in an independent manner (orders in lunch time, AI service during analytics bursts, etc.).



- **Compliance with constraints**: Good with PSU authentication and payment integration, and supports Saudi data and privacy requirements by centralized data access control.

3. How it is implemented in PSU FoodHub

Client Side (Mobile App)

- The following tasks are performed by the mobile app: visualization of the UI for the functionality of the app, ability to browse restaurants/menus, create an order, and track order, and view notifications.
- Mobile app utilizes HTTPS to make requests to the backend.
- The app uses a float-through authentication system, so passwords are not stored to the mobile app.

Server Side (Backend)

API Gateway: acts as a single entry point for the mobile app. performs routing of requests, rate limiting, and authentication checks.

Microservices (business services):

- User/Auth service (PSU authentication integration)
- Order service (create, update, track)
- Menu/Restaurant service (menus, availability, vendor info)
- Payment service (PSU payment integration)
- AI Predictor service (prep time and peak prediction)
- Notification service (push notifications)

Data Layer: shared storage of the database for users (minimally necessary), orders, restaurants, menus, and logs.

Software Design Documentation (SDD)

Detailed system design for PSU FoodHub system is presented in this phase. It includes the use of design patterns. This step transforms the previous architectural design phases into smaller, containable, maintainable, and scalable parts that can be implemented.

PSU FoodHub employs the use of design patterns that can help address the problem of system design complexity, and are helpful in implementing real-time system communication, and system flexible objects.

Considering the system requirements, the Client-Server (microservices) architecture, and describing quality attributes of performance, security, availability, and scalability, the following patterns were used:

- **Observer Pattern**: to provide support for real-time communication and notifications.
- **Facade Pattern**: this pattern is used to provide integration between a mobile application and backend services and simplify this interaction.
- **Builder Pattern**: this pattern is used for realizing a system through construction of a complex and flexible system of orders.

For each of the patterns employed design and communication, the structural and functional attributes of the system in order to successfully meet the requirements of the design patterns are outlined.

1. Design Pattern 1: Observer Pattern

Pattern Overview

When using The Observer Pattern, the state of one object affects many others. The object of which the state of is changing is referred to as the subject. The Observer Pattern automatically notifies all dependent objects, known as observers, of the subject's state change.

This pattern is beneficial for use in systems that require real-time updates for an event triggered pattern that requires immediate and continual interaction, often in pacing systems.

Structure

For PSU FoodHub, the following elements are included in the Observer Pattern:

- **Subject**: Order Service
- **Observers**:
    - Student mobile application (order tracking screen)

    - Notification service
    - Vendor dashboard
- **Event Trigger**: Change in order status (i.e. 'Pending', 'Preparing', 'Ready')

The Subject maintains many observers by notifying them of any state changes in the system.

Application Example in PSU FoodHub

One of the core functions of PSU FoodHub is to provide real-time update systems for order tracking and notifications, which is why The Observer Pattern is used.

When an order status is changed by a vendor to a state in the vendor dashboard:

1. The Order Service updates the order to one of the state changes
2. The Order Service triggers the notification to all observers
3. All of the observers receive the update immediately.
4. The mobile application of the student changes the tracking screen.
5. The Notification Service provides the user with a push

The Observer Pattern is used to provide immediate updates to the user without requiring an application refresh.

Justification

The selection of the Observer Pattern is used because:

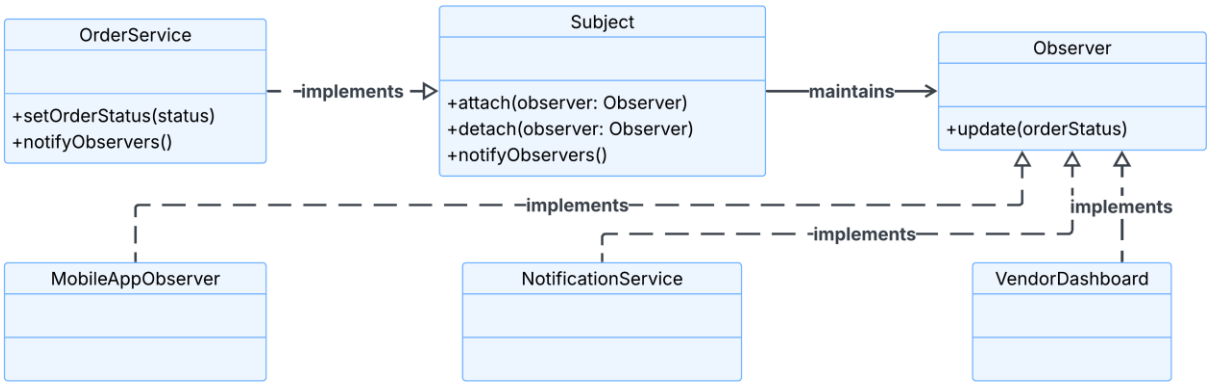
- The system requires immediate and real-time updates which are difficult using any other pattern (i.e. a polling pattern)
- It is event driven which has an architecture loosely coupled alignment with modern microservices design.
- It provides a decoupled model to the Order Service. This decoupled model is aligned with decoupled user interfaces and notifications.
- It can lead to better responsiveness and better overall user satisfaction.

Advantages

- Real-time communication can occur between system components
- Reduced coupling of system modules
- Improved extensibility of the system (addition of observers is easier)
- Event handling can scale

Disadvantages

- May increase system complexity as a consequence of asynchronous communication
- Event flow debugging may be harder
- Needs careful management of registering and de-registering observers



2. Design Pattern 2: Facade Pattern

Pattern Overview

The Facade Pattern is a structural design pattern which provides a simpler and easier interface to the larger, complicated, and intricate system composed of many subsystems. It conceals system complexity and provides the required functionality to the client.

Structure

In PSU FoodHub, the Facade Pattern contains:

- **Facade**: FoodHub Facade (or Order Management Facade)
- **Subsystems**:
  - Menu Service
  - Order Service
  - Payment Service
  - AI Predictor Service
  - Notification Service

The facades play the role of a bridge between the mobile frontend and the backend services.

Application Example in PSU FoodHub

The Facade Pattern serves the purpose of breaking down the complicated order placement process which consists of many services and steps.

In order for a student to place an order:

1. The application sends an order placement request to the FoodHub Facade.
2. The order placement request is verified, and menu items are validated against the order.
3. The estimated preparation and fulfillment of an order function is provided by the AI Predictor Service.
4. In order for the student to place an order, the transaction must first be completed by the Payment Service, and the order item must be fulfilled by the Order Service.
5. The request to inform the student of the fulfillment of the order is passed to the Notification Service.
6. The Facade returns the mobile application a simplified unified response, unlike the application that would have to individually and separately deal with the complicated interaction with the services.

Justification

There are many reasons for the choice of the Facade Pattern, such as:

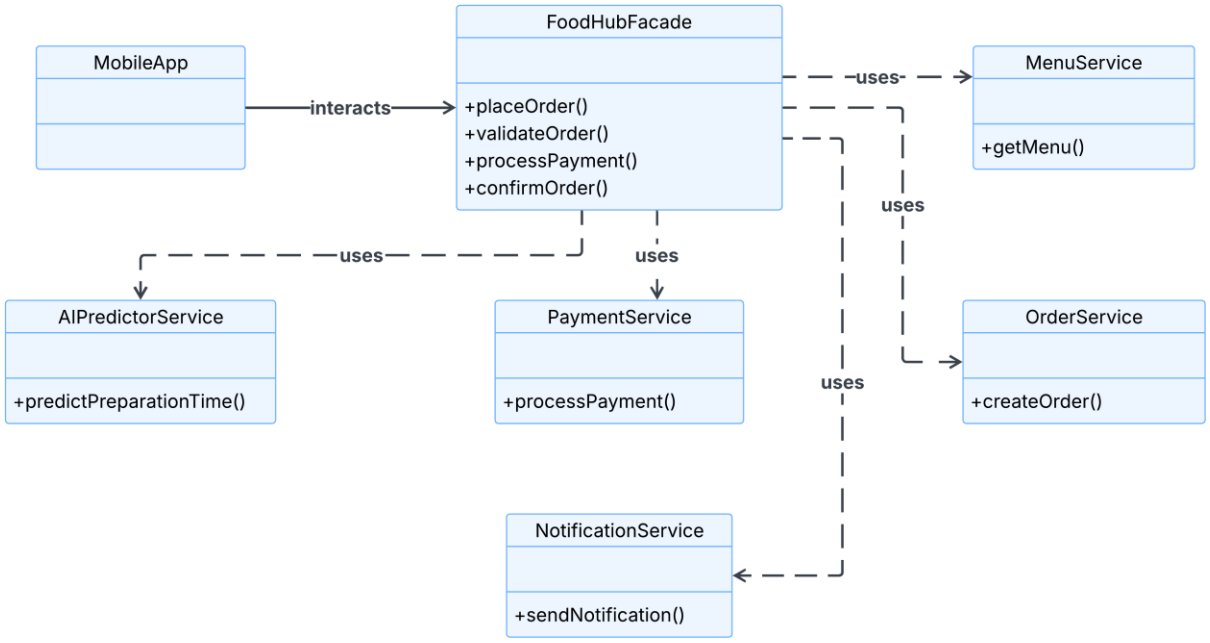
- The numerous component services that are interconnected.
- It simplifies the process of communicating between the frontend and backend.
- It decreases the number of the direct dependencies deployed.
- It increases the maintainability and readability of the system and codebase.
- It encourages the use of the API Gateway in the architecture.

Advantages

- Makes system interaction less complicated
- Lowers the amount of coupling
- Enhances the ease of keeping the code
- One complete interface to the user

Disadvantages

- May become too complex with too numerous responsibilities
- Could become, among other things, a single point of failure.



3. Design Pattern 3: Builder Pattern

Pattern Overview

The Builder Pattern allows you to develop a complex object a little at a time. This pattern enables you to develop different representations of the object through the same processes.

Structure

In PSU FoodHub, the Builder Pattern includes:

- **Product**: Order
- **Builder**: Order Builder
- **Director** (optional): Order Manager

The builder offers you the ability to develop an order object a bit at a time.

Application in PSU FoodHub

The Builder Pattern is used by the order customization feature in PSU FoodHub.

Food order customization lets users customize their food orders by specifying their food order.

- An order may include:
- Multiple food items
- Customizations (e.g., add/remove ingredients)
- Quantity selection
- Pickup time
- Payment method

The Order Builder lets the users:

- Add items to the order
- Apply customizations
- Set pickup time
- Select payment method
- Then, the order can be validated and become a final order.

It is therefore valid to supply the final order object in the system.

Justification

The Builder Pattern offers simplicity and advantages for the PSU FoodHub app in the following areas:

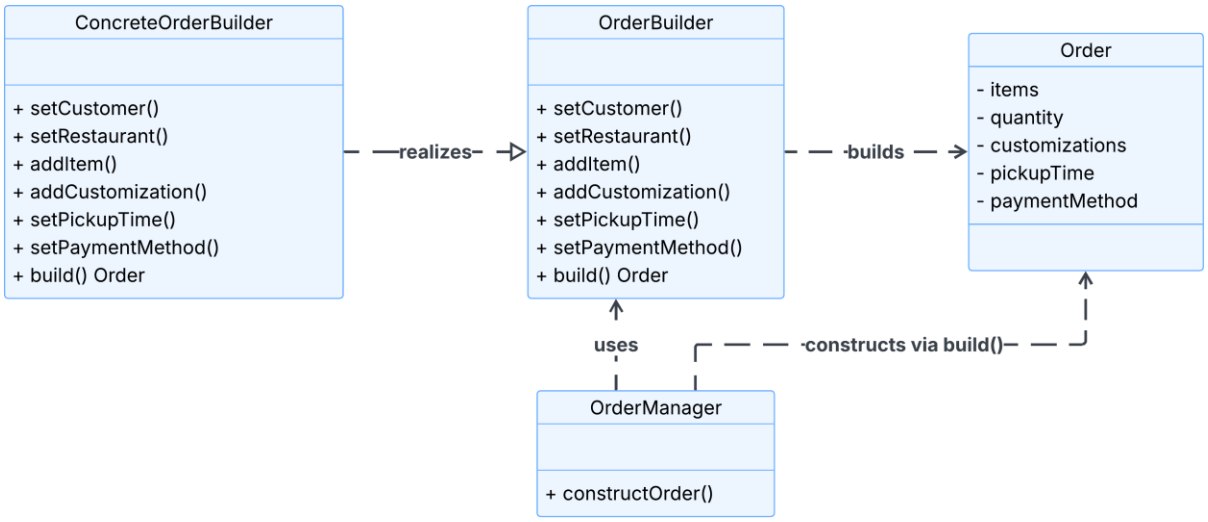
- Food orders and application orders can become customized with various attributes.
- Nobody likes to see complicated, random object structures.
- When the order customization feature is activated, the app is in an ideal state.

Advantages

- Obscures the complexity of the order customization feature
- Helps to keep the app clean and clear
- Enhances customization feature's flexibility
- Reduces potential for objects to state the incorrect information

Disadvantages

- Increased complexity and fragmentation of the order customization feature
- Unnecessary complexity for an app without order customization feature.
- Maintaining the app's proper state relies on the design.



Summary

This phase outlined the PSU FoodHub system's design by implementation of three design patterns: Observer, Facade, and Builder.

The design patterns were selected based on the system requirements.

- The Observer Pattern allows real-time interactions and notifications for the user.
- The Facade Pattern reduces user interactions with the complex backend and improves the system's usability and maintainability.
- The Builder Pattern is the most flexible and optimal way for the user to construct complex order objects.

The implementation of the above-listed design patterns improves the modularity, scalability, and maintainability of the PSU FoodHub. The above-described design patterns will ensure that PSU FoodHub effectively manages real-world use cases and will be able to scale to meet unexpected demands.